

FGV EMap
Thalis Ambrosim Falqueto

Engenharia de Software

Holy emapian

Rio de Janeiro
2026

Conteúdo

1	Aula 1 - Documentação, Contratos e SRP	3
1.1	Cacto	3
1.2	Por que o processar deveria ser vários métodos (SRP)	7
1.3	Loja (exercício de correção)	7
1.4	Termos da Aula 1	9
2	Aula 2 - Simple Factory, Factory Method e OCP	9
2.1	Simple Factory	10
2.2	Factory Method	11
2.3	Termos da Aula 2	12
3	Aula 3 - Builder e Singleton	13
3.1	Builder	13
3.2	Singleton	16
3.2.1	Singleton no __new__	16
3.2.2	Singleton no decorador	17
3.3	Termos da Aula 3	18
4	quinta	18

Aula 1 - Documentação, Contratos e SRP

A aula usa dois exemplos: a “Cacto” é simplesmente uma loja de cactos com um método de quase 90 linhas que faz de tudo (aparentemente feito por algum aluno de LP), sem nenhuma documentação, e a “Loja” é o exercício de correção, que isola só a fatia de “calcular o frete” desse mesmo problema pra refatorar passo a passo, do jeito errado até o SRP resolvido.

1.1 Cacto

A loja vende cactos — cada um com nome, preço e altura — e monta pedidos com vários itens:

```
1 class Cacto:
2     def __init__(self, nome, preco, altura_cm):
3         self.nome = nome
4         self.preco = preco
5         self.altura_cm = altura_cm
6
7
8 class ItemPedido:
9     def __init__(self, cacto, quantidade):
10        self.cacto = cacto
11        self.quantidade = quantidade
```

Exemplo de **design by contract** citado em aula: o `__init__` de `ItemPedido` já declara, o que consome pra funcionar — um `cacto` e uma `quantidade`. Só que, como o próprio comentário no código admite, nada garante que `cacto` é de fato um `Cacto`. Documentar com docstring é pedir pra quem for usar a classe seguir o combinado: Python é fracamente tipado, então nada impede alguém de passar qualquer coisa ali. A saída ingênua seria verificar o tipo dentro do `__init__`, mas fazer esse tipo de verificação em todos os métodos e todos os parâmetros deixa o programa lento. Aí que entra o termo dito pelo Pinho: você declara o comportamento esperado e confia que quem chamou respeitou, em vez de reverificar tudo o tempo todo.

Na média, tudo que é interno ao sistema é tratado assim, e a verificação de fato só entra quando o sistema recebe algo de fora dele. Se um contrato não é respeitado, a consequência pode ser tão grave quanto o caso citado em aula do foguete da NASA que explodiu (não explodiu, mas **ver**).

Continuando o código, o pedido guarda os dados do cliente e a lista de itens:

```
1 class Pedido:
2     def __init__(self, cliente, email, cep, tipo_entrega, cupom, numero_cartao):
3         self.cliente = cliente
4         self.email = email
5         self.cep = cep
6         self.tipo_entrega = tipo_entrega
7         self.cupom = cupom
8         self.numero_cartao = numero_cartao
9         self.itens = []
```

E a entrega tem quatro formas possíveis, todas implementando o mesmo contrato abstrato:

```
1 class Entrega(ABC):
```

```
2     @abstractmethod
3     def calcular_frete(self, peso_kg, cep):
4         ...
5
6     @abstractmethod
7     def prazo_em_dias(self, cep):
8         ...
9
10    @abstractmethod
11    def codigo_rastreio(self):
12        ...
13
14
15    class Sedex(Entrega):
16        def calcular_frete(self, peso_kg, cep):
17            base = 18.90 + peso_kg * 2.35
18            if cep.startswith("6") or cep.startswith("7"):
19                base = base * 1.4
20            return base
21
22        def prazo_em_dias(self, cep):
23            return 2 if cep.startswith("0") else 5
24
25        def codigo_rastreio(self):
26            return "BR314159265BR"
27
28
29    class Drone(Entrega):
30        def calcular_frete(self, peso_kg, cep):
31            base = 49.90 + peso_kg * 8.0
32            if peso_kg > 3.0:
33                base = base + 60.0
34            return base
35
36        def prazo_em_dias(self, cep):
37            return 1
38
39        def codigo_rastreio(self):
40            return "DRN-8080"
41
42
43    class PomboCorreio(Entrega):
44        def calcular_frete(self, peso_kg, cep):
45            return 4.50 + peso_kg * 0.75
46
47        def prazo_em_dias(self, cep):
```

```

48         return
49
50     def codigo_rastreio(self):
51         return "OLHE-PARA-O-CEU"
52
53
54     class CarrocaDeBoi(Entrega):
55         def calcular_frete(self, peso_kg, cep): # nao usa o cep
56             return 9.90 + peso_kg * 0.40
57
58         def prazo_em_dias(self, cep): # nao usa o cep
59             return 30
60
61         def codigo_rastreio(self):
62             return "BOI-0404"

```

Tudo isso converge pro método onde o professor usa os termos como **code smell** e **god class**: `ProcessadorDePedido.processar` calcula subtotal, aplica desconto, calcula peso, decide a entrega, calcula frete e imposto, valida cartão, grava em “banco”, manda e-mail, imprime nota fiscal e loga — tudo junto, num só lugar:

```

1  class ProcessadorDePedido:
2      def processar(self, pedido):
3          subtotal = 0
4          for item in pedido.itens:
5              subtotal = subtotal + item.cacto.preco * item.quantidade
6
7          desconto = 0
8          if pedido.cupom is not None:
9              if pedido.cupom == "NULLSAFE10":
10                 desconto = subtotal * 0.10
11                 elif pedido.cupom == "OFFBYONE5":
12                     desconto = 5.0
13                 elif pedido.cupom == "HELLOWORLD":
14                     desconto = subtotal * 0.15
15                     if desconto > 40.0: # if devia ta pra fora desse elif
16                         desconto = 40.0
17
18             peso_kg = 0
19             for item in pedido.itens:
20                 peso_kg = peso_kg + item.quantidade * (item.cacto.altura_cm * 0.05)
21
22             if pedido.tipo_entrega == "SEDEX":
23                 entrega = Sedex()
24             elif pedido.tipo_entrega == "DRONE":
25                 entrega = Drone()
26             elif pedido.tipo_entrega == "POMBO":

```

```

27         entrega = PomboCorreio()
28     elif pedido.tipo_entrega == "CARROCA":
29         entrega = CarrocaDeBoi()
30     else:
31         entrega = Sedex() # talvez o sedex n entregue nesse lugar, tem q
                             verificar
32
33     frete = entrega.calcular_frete(peso_kg, pedido.cep)
34     base_imposto = subtotal - desconto
35     imposto = base_imposto * 0.18
36     total = base_imposto + frete + imposto
37
38     if pedido.numero_cartao is None or len(pedido.numero_cartao) != 16:
39         raise ValueError("cartao invalido")
40     soma = 0
41     for c in pedido.numero_cartao:
42         if c < "0" or c > "9":
43             raise ValueError("cartao invalido")
44         soma = soma + int(c)
45     if soma % 10 != 0:
46         raise ValueError("cartao recusado")
47
48     sql = (
49         "INSERT INTO pedidos (cliente, cep, entrega, total) VALUES ('"
50         + pedido.cliente + "', '" + pedido.cep + "', '" + pedido.tipo_entrega
51         + "', " + f"{total:.2f}" + ")"
52     )
53     print("[BANCO] " + sql)
54     for item in pedido.itens:
55         print(
56             "[BANCO] INSERT INTO itens (pedido_cliente, cacto, qtd) VALUES ('"
57             + pedido.cliente + "', '" + item.cacto.nome + "', "
58             + str(item.quantidade) + ")"
59         )
60
61     corpo = "<html><body>"
62     corpo = corpo + "<h1>Obrigado, " + pedido.cliente + "!</h1>"
63     corpo = corpo + "<p>Seus cactos foram compilados sem warnings e entraram na
                             fila de deploy.</p><ul>"
64     for item in pedido.itens:
65         corpo = corpo + "<li>" + str(item.quantidade) + "x " + item.cacto.nome
                             + "</li>"
66     corpo = corpo + "</ul><p>Frete: R$ " + f"{frete:.2f}" + "</p>"
67     corpo = corpo + "<p>Total: R$ " + f"{total:.2f}" + "</p>"
68     corpo = corpo + "<p>Previsao de entrega: " +
                             str(entrega.prazo_em_dias(pedido.cep)) + " dias</p>"
69     corpo = corpo + "</body></html>"

```

```

70     print("[SMTP] enviando para " + pedido.email)
71     print(corpo)
72
73     print("=== NOTA FISCAL ELETRONICA ===")
74     print("DESTINATARIO: " + pedido.cliente)
75     print("BASE DE CALCULO: " + f"{base_imposto:.2f}")
76     print("ICMS 18%: " + f"{imposto:.2f}")
77     print("VALOR TOTAL: " + f"{total:.2f}")
78     print("=====")
79
80     print(
81         "[LOG] pedido de " + pedido.cliente + " processado com "
82         + str(len(pedido.itens)) + " itens, total " + f"{total:.2f}"
83     )
84
85     return total

```

Pinho também disse que ‘quanto mais você precisa dar scroll numa função, pior ela é’, ‘código longo com variáveis pouco significativas é ruim de manter’ e é ‘pouco provável que você realmente precise de uma classe com 1000 linhas’ (fazendo referência a um código em produção real).

1.2 Por que o processar deveria ser vários métodos (SRP)

O problema dessa função é que ela tem várias (funções). Se listássemos, quem, na empresa, poderia pedir uma mudança em `ProcessadorDePedido.processar` — e apontando exatamente onde, dentro do método, cada um bateria:

- a **contabilidade** precisar mudar a alíquota — o `0.18` fixo em `imposto = base_imposto * 0.18`;
- o **marketing** criar um cupom novo — o bloco `if pedido.cupom == "NULLSAFE10": ...`
- o **marketing** precisar mudar o e-mail — o bloco que monta `corpo` em HTML e manda pro “SMTP”;
- o **gateway do cartão** mudar — a validação de `numero_cartao` (tamanho, dígitos, soma);
- o **DBA** precisar mudar uma coluna — o `sql = "INSERT INTO pedidos ..."` montado por concatenação de string (que, à parte da aula, também é uma porta aberta pra SQL injection, já que `pedido.cliente` entra direto na query sem tratamento nenhum);
- o **SEFAZ** mudar o layout da nota — o bloco `"=== NOTA FISCAL ELETRONICA ==="`;
- os **Correios** mudarem o modo de calcular o peso — a fórmula `peso_kg = peso_kg + item.quantidade * (item.cacto.altura_cm * 0.05)`;
- o **COO** querer oferecer outra forma de envio — o `if/elif` de `tipo_entrega`.

É esse problema que o Princípio de Responsabilidade Única (SRP) corrige: cada componente deve apresentar uma única responsabilidade. É o primeiro dos cinco princípios do SOLID.

1.3 Loja (exercício de correção)

O professor exemplifica a solução numa loja simplificada que faz somente o frete, produto e recibo, pra refatorar passo a passo. A etapa 1 comete o mesmo erro numa escala menor:

```

1  class Loja:
2      def processar(self, cliente, valor, peso_kg, centro):
3          if centro == 'Sao Paulo':
4              frete = 10.0 + 2.0 * peso_kg

```

 Python

```

5         elif centro == 'Manaus':
6             frete = 25.0 + 1.2 * peso_kg
7         else:
8             frete = 10.0 + 2.0 * peso_kg
9
10        total = valor + frete
11        return f'{cliente} pagou R$ {total:.2f} (envio de {centro})'

```

A etapa 2 divide Loja1 em peças, cada uma cuidando de uma única coisa — calcular o total, gerar o recibo, e decidir o custo do frete, com uma subclasse por tipo de frete:

```

1  class Frete(ABC):
2      @abstractmethod
3      def custo(self, peso_kg):
4          ...
5
6  class FreteRodoviario(Frete):
7      def custo(self, peso_kg):
8          return 10.0 + 2.0 * peso_kg
9
10 class FreteFluvial(Frete):
11     def custo(self, peso_kg):
12         return 25.0 + 1.2 * peso_kg
13
14 class CalculadoraTotal:
15     def total(self, valor, frete):
16         return valor + frete
17
18 class Recibo:
19     def gerar(self, cliente, total, centro):
20         return f'{cliente} pagou R$ {total:.2f} (envio de {centro})'
21
22
23 class Loja2:
24     def __init__(self, calculadora, recibo):
25         self.calculadora = calculadora
26         self.recibo = recibo
27
28     def processar(self, cliente, valor, peso_kg, centro):
29         if centro == 'Sao Paulo':
30             frete = FreteRodoviario()
31         elif centro == 'Manaus':
32             frete = FreteFluvial()
33         else:
34             frete = FreteRodoviario()
35
36         total = self.calculadora.total(valor, frete.custo(peso_kg))

```

 Python

```
37
38         return self.recibo.gerar(cliente, total, centro)
```

Agora, Loja2 ainda tem um if, decidindo qual objeto usar, mas não faz mais o cálculo do frete nem monta o recibo. Rodando as duas etapas lado a lado, o resultado é o mesmo, mas a segunda já está pronta pra crescer sem precisar reescrever tudo de novo:

```
1  cliente = 'Ada'
2  valor = 100
3  peso_kg = 2
4
5  print('etapa 1 - tudo cagado')
6  loja1 = Loja1()
7  print(loja1.processar(cliente, valor, peso_kg, 'Sao Paulo'))
8  print(loja1.processar(cliente, valor, peso_kg, 'Manaus'))
9
10 print('etapa 2 - SRP dominado')
11 loja2 = Loja2(CalculadoraTotal(), Recibo())
12 print(loja2.processar(cliente, valor, peso_kg, 'Sao Paulo'))
13 print(loja2.processar(cliente, valor, peso_kg, 'Manaus'))
```

 Python

1.4 Termos da Aula 1

- **Docstring, typehint** - Vocês já sabem isso;
- **Design by contract** — declarar o comportamento esperado de um componente (o que ele espera receber, o que garante devolver) e confiar nisso, em vez de reverificar tudo em tempo de execução.
- **Defeito (fault/bug)** — falha concreta e localizável no código-fonte, que existe independente de ser executada.
- **Falha (failure)** — comportamento incorreto observável quando o programa roda e um defeito é de fato acionado.
- **Code smell** — sintoma de que o design do código está ruim, mesmo funcionando sem erros; o “cheiro” de um problema estrutural.
- **God class** — classe (ou método) que sabe e faz coisas demais, concentrando responsabilidades que deveriam estar separadas.
- **SRP (Princípio de Responsabilidade Única)** — cada componente deve ter um único motivo pra mudar; o S do SOLID.

Aula 2 - Simple Factory, Factory Method e OCP

A solução da Loja2 (Section 1.3) resolveu o SRP, mas o `if centro == ...` continua dentro de `processar`, decidindo qual `Frete` instanciar ainda hardcoded, ainda fazendo verificação de string, o que ainda é problema mesmo depois do SRP resolvido (Professor disse que deixar essas verificações em string é um crime, mas não vem ao caso). Some a isso uma regra de negócio nova: a loja quer simular o frete antes da compra, não só calculá-lo depois que o cliente já decidiu comprar, e passa a atender mais centros de distribuição. Dessa forma, agora, vários lugares diferentes vão precisar da mesma lógica de “qual frete usar para este centro”, e ela está presa dentro de `Loja2`.

2.1 Simple Factory

Simple Factory resolve o problema das classes: tira o if de dentro da loja e concentra numa única classe. A ideia, resumida em aula, é simplesmente mudar o if de lugar:

```
1  class FabricaDeFrete:
2      def criar(self, centro):
3          if centro == 'Sao Paulo':
4              return FreteRodoviario()
5          elif centro == 'Manaus':
6              return FreteFluvial()
7          else:
8              return FreteRodoviario()
9
10
11 class SimuladorDeFrete:
12     def __init__(self, fabrica):
13         self.fabrica = fabrica
14
15     def simular(self, centro, peso_kg):
16         return self.fabrica.criar(centro).custo(peso_kg)
17
18
19 class Loja3:
20     def __init__(self, fabrica, calculadora, recibo):
21         self.fabrica = fabrica
22         self.calculadora = calculadora
23         self.recibo = recibo
24
25     def processar(self, cliente, valor, peso_kg, centro):
26         frete = self.fabrica.criar(centro)
27         total = self.calculadora.total(valor, frete.custo(peso_kg))
28         return self.recibo.gerar(cliente, total, centro)
```

Agora, a loja não precisa mais entender de frete. O ganho aparece em isolar o if do resto do código: o acoplamento entre Loja e a lógica de frete diminui. Pinho disse isso numa frase: ‘toda vez que uma mudança no código não obriga o main a mudar junto, o design melhorou’. De fato, a etapa 3 do main só precisa montar a fábrica e injetá-la:

```
1  print("ETAPA 3 - SRP simple factory")
2  fabrica = FabricaDeFrete()
3  loja3 = Loja3(fabrica, CalculadoraTotal(), Recibo())
4  print(loja3.processar(cliente, valor, peso_kg, 'Sao Paulo'))
5  print(loja3.processar(cliente, valor, peso_kg, 'Manaus'))
6  custo = SimuladorDeFrete(fabrica).simular("Belém", peso_kg)
7
8  print("Simulação Belém", custo)
```

Note que essa última linha ainda esconde o mesmo bug do else. Como `FabricaDeFrete.criar` só reconhece 'Sao Paulo' e 'Manaus', simular pra “Belém” cai no else e devolve `FreteRodoviario`, o que pode não ser o certo pro caso.

Nesse ponto a aula sai do código e coloca o Simple Factory dentro de um vocabulário maior de arquitetura. O professor ajuda a definir alguns termos, que vou definir melhor aqui, não exatamente do jeito dito: um **padrão de projeto** é uma saída clássica, já testada, para um problema recorrente de modelagem; a referência dada foi o livro de 1994 do **GoF** (Gang of Four).

O GoF organiza os 23 padrões do livro de 1994 em três famílias: **padrões criacionais** (como criar objetos — Factory Method, Abstract Factory, Builder, Prototype, Singleton), **padrões estruturais** (como compor classes e objetos — Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) e **padrões comportamentais** (como objetos interagem e distribuem responsabilidade — Observer, Strategy, State, Template Method, entre outros). O Simple Factory é um idioma didático, usado como degrau pra chegar no Factory Method — esse sim um dos criacionais reconhecidos pelo GoF.

2.2 Factory Method

O Simple Factory ainda tem um if central, só que concentrado num único lugar. Em vez de uma fábrica perguntar “qual centro é esse?”, cada centro de distribuição sabe, por si mesmo, qual frete criar — quem entrega essa decisão ao código passa a ser a própria classe do centro de distribuição. Isso é feito com um método abstrato que cada subclasse implementa à sua maneira, uma por centro (faz isso importando o decorador e passando o decorador a função e a classe a própria factory - é um método declarado sem corpo ou código de execução, que serve como uma regra obrigatória para que as classes filhas criem sua própria implementação). Vejamos no exemplo:

```
1  class CentroDeDistribuicao(ABC):
2      def __init__(self, cidade, calculadora, recibo):
3          self.cidade = cidade
4          self.calculadora = calculadora
5          self.recibo = recibo
6
7      @abstractmethod
8      def criar_frete(self):
9          ...
10
11     def despachar(self, cliente, valor, peso_kg):
12         frete = self.criar_frete()
13         total = self.calculadora.total(valor, frete.custo(peso_kg))
14         return self.recibo.gerar(cliente, total, self.cidade)
15
16
17 class CentroSaoPaulo(CentroDeDistribuicao):
18     def __init__(self, calculadora, recibo):
19         super().__init__("São Paulo", calculadora, recibo)
20
21     def criar_frete(self):
22         return FreteRodoviario()
```

 Python

```

23
24
25 class CentroManaus(CentroDeDistribuicao):
26     def __init__(self, calculadora, recibo):
27         super().__init__("Manaus", calculadora, recibo)
28
29     def criar_frete(self):
30         return FreteFluvial()
31
32
33 class CentroBelem(CentroDeDistribuicao):
34     def __init__(self, calculadora, recibo):
35         super().__init__("Belém", calculadora, recibo)
36
37     def criar_frete(self):
38         return FreteFluvial()

```

Agora, Belém tem, por conta própria, o frete fluvial certo — sem precisar tocar em nenhum `if`. O restante do fluxo (calcular total, gerar recibo) fica implementado uma única vez, no método concreto despachar da classe-base — as subclasses só precisam dizer qual frete usar. O polimorfismo (significa “muitas formas” (no caso, a função `criar_frete` é a polimórfica)) substitui o `if`: quando o código chama `self.criar_frete()`, quem responde já é o objeto do centro certo, decidido no momento em que a classe foi escolhida (a instancição), não dentro de um `if` em tempo de execução. Essa é a etapa 4:

```

1 print("ETAPA 4")
2 sao_paulo = CentroSaoPaulo(CalculadoraTotal(), Recibo())
3 manaus = CentroManaus(CalculadoraTotal(), Recibo())
4 belem = CentroBelem(CalculadoraTotal(), Recibo())
5 print(sao_paulo.despachar(cliente, valor, peso_kg))
6 print(manaus.despachar(cliente, valor, peso_kg))
7 print(belem.despachar(cliente, valor, peso_kg))

```

 Python

O mesmo raciocínio vale pra produtos: pra cada produto novo que precise de fábrica própria, o padrão pede duas classes, o produto e a fábrica do produto (aqui, o “produto” é o Frete, e a “fábrica” é o próprio `CentroDeDistribuicao`).

Comparado ao Simple Factory, adicionar um centro de distribuição novo passa a significar **criar** uma subclasse, e não editar nenhuma existente: nada do código antigo precisou mudar pra isso, nem o `CentroDeDistribuicao` já existente precisou ser tocado pra nascer um tipo novo. Esse é, literalmente, o enunciado do Open/Closed Principle, o O do SOLID: uma classe deve estar fechada para modificação e aberta para extensão.

2.3 Termos da Aula 2

- **Simple Factory** — classe cujo único trabalho é decidir qual objeto concreto criar, tirando essa decisão de quem consome o objeto.
- **Padrão de projeto (design pattern)** — solução clássica e já testada pra um problema recorrente de modelagem orientada a objetos; catalogados no livro do GoF (1994).
- **GoF (Gang of Four)** — apelido dos quatro autores do livro **Design Patterns** (1994).

- **Padrão criacional** — categoria de padrão de projeto que resolve “qual é a forma correta de criar um objeto” (Simple Factory, Factory Method, Builder, Singleton).
- **Factory Method** — cada subclasse decide, via método abstrato sobrescrito, qual objeto concreto criar; substitui o if por polimorfismo.
- **Polimorfismo** — objetos de subclasses diferentes respondem à mesma chamada de método, cada um à sua maneira.
- **OCP (Open/Closed Principle)** — uma classe deve estar fechada para modificação e aberta para extensão; o O do SOLID.

Aula 3 - Builder e Singleton

3.1 Builder

O padrão de criação Builder fala sobre esconder a complexidade de montar um (ou vários (do mesmo)) objeto. O exemplo dado foi fazer uma requisição HTTP (você precisa de url, header, token, body), ou uma consulta SQL via ORM, com schema, select, from, where: linhas e mais linhas só pra montar uma chamada, o que não é gerenciável e dificulta a leitura pra quem lê depois.

Para resolver isso, cria-se uma classe específica só pra construir objetos de um tipo (nomeada normalmente como {Produto}Builder), onde cada decisão de criação complexa fica dentro dessa classe, e o objetivo é fazer decisões encadeadas (retornando o self a cada método do Builder).

O exemplo de aula foi montar um computador:

```
1  class Computador:
2      def __init__(self):
3          self.processador = None
4          self.memoria_gb = None
5          self.armazenamento_gb = None
6          self.placa_video = None
7
8      def __str__(self):
9          return f'Processador: {self.processador}, Memória: {self.memoria_gb},
          Armazenamento: {self.armazenamento_gb} e Placa de Vídeo {self.placa_video}'
10
11
12  class ComputadorBuilder:
13      def __init__(self):
14          self.computador = Computador()
15
16      def com_processador(self, processador):
17          self.computador.processador = processador
18          return self
19
20      def com_memoria_gb(self, memoria_gb):
21          self.computador.memoria_gb = memoria_gb
22          return self
23
24      def com_armazenamento_gb(self, armazenamento_gb):
```

```

25     self.computador.armazenamento_gb = armazenamento_gb
26     return self
27
28     def com_placa_video(self, placa_video):
29         self.computador.placa_video = placa_video
30         return self
31
32     def construir(self):
33         return self.computador
34
35
36 computador_gamer = (
37     ComputadorBuilder()
38     .com_processador('Intel I7')
39     .com_memoria_gb('16 GB RAM')
40     .com_armazenamento_gb('1024')
41     .com_placa_video('Nvd 5070')
42     .construir()
43 )

```

Cada propriedade de construção difícil vira um método que devolve `self`, enquanto o resto fica no `__init__` de `Computador`. Todos os métodos do builder fazem isso, com exceção do `.construir()`, que é quem devolve o produto final já pronto. Na vida real você raramente monta um computador atributo por atributo toda vez: normalmente existe uma classe com as construções comuns já prontas (configurações padrão), reaproveitando o mesmo builder por baixo:

```

1  # na vida real criamos objetos padrão
2  # muitas vezes vamos ter uma classe com todas as construções comuns que precisamos
   (configurações prontas)
3  class InfoCentro:
4      def montar_computador_basico(self):
5          return (
6              ComputadorBuilder()
7              .com_processador('Intel I3')
8              .com_memoria_gb('8 GB RAM')
9              .com_armazenamento_gb('256')
10             .com_placa_video('Integrada')
11             .construir()
12         )
13
14     def montar_computador_gamer(self):
15         return (
16             ComputadorBuilder()
17             .com_processador('Intel I7')
18             .com_memoria_gb('16 GB RAM')
19             .com_armazenamento_gb('1024')
20             .com_placa_video('Nvd 5070')

```

```
21         .construir()  
22     )
```

Para esse exemplo, fiquei com a dúvida: qual a utilidade do Builder se eu podia simplesmente instanciar Computador passando as especificações direto (ou, se precisasse, colocar a classe Computador diretamente dentro de uma classe principal (como a InfoCentro))? Nesse caso, a resposta é: nenhuma, ou quase. Computador(processador='Intel I7', memoria_gb='16 GB RAM', armazenamento_gb='1024', placa_video='Nvd 5070') faz exatamente o mesmo que as cinco linhas encadeadas do ComputadorBuilder, com menos código e sem precisar de uma classe extra. O motivo é que o Builder clássico nasceu pra resolver o problema do construtor telescópico, em que pular um parâmetro opcional te obrigava a passar null pra todos os anteriores na ordem certa. Nós não conseguimos enxergar esse problema pois o Python já resolve isso de fábrica com kwargs e parâmetros nomeados (colocar parametro = valor_padrao).

O Builder só ganha da alternativa direta quando o que está sendo montado não é só um punhado de campos escalares: é o caso dos dois exemplos que abriram a aula, montar um request HTTP ou uma query SQL, onde você vai **acumulando** uma quantidade variável de coisas (headers, cláusulas where) em vez de preencher campos fixos, ou quando um passo depende do resultado de outro (validar se a fonte de energia aguenta a placa de vídeo escolhida, por exemplo), ou quando você quer impedir que um objeto pela metade escape pro resto do código.

Um exemplo concreto onde o Builder faz sentido, voltando pro domínio da Cacto: montar a consulta de pedidos com um número variável de filtros, onde cada .com_X() só entra na query se o chamador realmente pedir aquele filtro.

```
1  class ConsultaPedidosBuilder:
2      def __init__(self):
3          self.tabela = "pedidos"
4          self.condicoes = []
5          self.parametros = []
6
7      def com_cliente(self, cliente):
8          self.condicoes.append("cliente = %s")
9          self.parametros.append(cliente)
10         return self
11
12     def com_tipo_entrega(self, tipo_entrega):
13         self.condicoes.append("tipo_entrega = %s")
14         self.parametros.append(tipo_entrega)
15         return self
16
17     def com_total_minimo(self, total_minimo):
18         self.condicoes.append("total >= %s")
19         self.parametros.append(total_minimo)
20         return self
21
22     def construir(self):
23         sql = f"SELECT * FROM {self.tabela}"
24         if self.condicoes:
```

 Python

```

25         sql += " WHERE " + " AND ".join(self.condicoes)
26         return sql, self.parametros
27
28
29 sql, parametros = (
30     ConsultaPedidosBuilder()
31     .com_tipo_entrega("DRONE")
32     .com_total_minimo(100.0)
33     .construir()
34 )
35 # sql = 'SELECT * FROM pedidos WHERE tipo_entrega = %s AND total >= %s'
36 # parametros = ['DRONE', 100.0]

```

Um kwarg fixo não dá conta disso de forma limpa — teria que ser algo como `consultar(cliente=None, tipo_entrega=None, total_minimo=None)` com um `if` pra cada parâmetro checando se é `None` antes de colar no SQL, o que é exatamente a bagunça que o Builder organiza aqui. E de quebra, essa versão já resolve o problema de SQL injection apontado lá na Cacto: em vez de concatenar `pedido.cliente` direto na string, os valores viram parâmetros separados (`%s` + lista `parametros`), passados pro driver do banco em vez de virarem texto da query.

3.2 Singleton

Singleton parte de um problema oposto: em vez de facilitar criar vários objetos, ele impede que exista mais de um. Só podemos ter uma instância na aplicação, impossível ter um segundo objeto. A analogia dada em aula: é como colocar um assento onde só cabe uma pessoa. Um nome comum pra esse tipo de classe é `Config`, embora o exemplo em si não implemente nenhum método de negócio, só o mecanismo.

3.2.1 Singleton no `__new__`

A primeira forma de resolver isso funciona em qualquer linguagem orientada a objetos (existe outro jeito, mas só funciona em Python) e mexe direto no `__new__`:

```

1  from typing import ClassVar, Self
2
3
4  class SingletonNew:
5      _instance: ClassVar[SingletonNew | None] = None
6
7      def __new__(cls) -> Self:
8          if cls._instance is None:
9              print('[SingletonNew] Criando Nova instância')
10             cls._instance = super().__new__(cls)
11          else:
12              print('[SingletonNew] Retornando instância existente!')
13
14             return cls._instance
15
16      def __init__(self):
17          self.data: str = 'Shared Resource'

```

 Python

```
18
19
20 print(SingletonNew() is SingletonNew())
```

Essa solução também pode ser misturada com Builder e outros padrões — o código acima é só o esqueleto, poderia ter métodos de negócio como qualquer classe normal. Só que, como dito pelo Pinho, a solução não é boa: toda vez que você instancia `SingletonNew()`, o `__init__` roda de novo mesmo quando `__new__` está devolvendo a instância antiga — o estado se torna repetido sempre, mesmo sem criar um objeto novo. E pior: com herança, o Singleton se duplica. Se uma classe filha herda de uma classe mãe que já implementa esse `__new__`, a classe filha acaba pegando o singleton da classe mãe. Não dá pra usar essa estratégia no `__new__` de forma confiável quando existe hierarquia.

3.2.2 Singleton no decorador

A segunda forma tenta resolver isso com um decorador, guardando cada instância criada num dicionário — em vez de uma cadeira, um banco com vários lugares, um pra cada tipo de classe decorada:

```
1  from collections.abc import Callable
2  from typing import Any
3
4
5  def singleton_naive[T](cls: type[T]) -> Callable[..., T]:
6      instances: dict[type[T], T] = {}
7
8      def get_instance(*args: Any, **kwargs: Any) -> T:
9          if cls not in instances:
10             instances[cls] = cls(*args, **kwargs)
11             return instances[cls]
12
13     return get_instance
14
15
16 @singleton_naive
17 class SingletonNaive:
18     '''Singleton sem uma preocupação terrível'''
19
20     def __init__(self, rotulo: str = 'Sem algo que seria útil aqui') -> None:
21         self.rotulo = rotulo
22
23
24 print(type(SingletonNaive))
25 print(SingletonNaive().__name__, SingletonNaive().__doc__)
```

O problema aqui é de outra natureza: `get_instance` deveria receber tudo que o construtor da classe original receberia, mas como o decorador funciona pra qualquer classe, ele não tem como saber de antemão o que cada uma espera. E tem um efeito colateral mais sério — depois do decorador, `SingletonNaive` deixa de ser uma classe e passa a ser uma função (`get_instance`, guardada como

closure, com `cls` capturado como `freevar` no frame da função original). Isso significa que um `isinstance` contra `SingletonNaive` já não funciona mais do jeito esperado, porque você estaria comparando contra uma função, não contra um tipo. O singleton em si funciona perfeitamente, mas a classe não funciona mais perfeitamente como classe.

3.3 Termos da Aula 3

- **Builder** — padrão criacional que isola a lógica de montar um objeto complexo numa classe própria, com métodos encadeados.
- **Singleton** — padrão criacional que garante que uma classe tenha, no máximo, uma única instância na aplicação.
- **Closure** — função que “lembra” variáveis do escopo onde foi definida, mesmo depois que esse escopo termina de executar.
- **Freevar (variável livre)** — variável usada dentro de uma função mas definida fora dela, capturada pela closure.

quinta

quero fazer um widget de calendario e clima criando fabrica de elemento de interface fabrica concreta das fabricas separa em ipad e ios ao escolher uma fabrica, todo o restante do programa se adequa a ela geração de widgets de acordo com osistema operacional mas ainda tá desconexo aqui tem uns ifs fabricas concretas de cada familia todos os produtos de todas as familias são tratados como genéricos

```
1  """
2  A Abstract Factory (Fábrica Abstrata) é um padrão de projeto criacional que
3  produzir famílias de objetos relacionados ou dependentes sem especificar suas
4  classes concretas.
5  Importante:
6  As classes de Abstract Factory frequentemente são baseadas em um conjunto de
7  Factory Methods, mas também é possível usar Prototype para compor os métodos
8  dessas classes. Mas, como a turma conhece Factory Methods, vou seguir esta
9  abordagem
10 """
11 from abc import ABC, abstractmethod
12
13 class UIFactory(ABC):
14     @abstractmethod
15     def create_calendar(self): ...
16
17     @abstractmethod
18     def create_weather(self): ...
19
20 class iPadOSFactory(UIFactory):
21     def create_calendar(self):
22         print("Factory da Plataforma iPadOS: Calendário")
```

```

21     iPadOSCalendarWidget().create_calendar()
22
23     def create_weather(self):
24         print("Factory da Plataforma iPadOS: Clima")
25         iPadOSWeatherWidget().create_weather()
26
27     class iOSFactory(UIFactory):
28         def create_calendar(self):
29             print("Factory da Plataforma iOS: Calendário")
30             iOSCalendarWidget().create_calendar()
31
32         def create_weather(self):
33             print("Factory da Plataforma iOS: Clima")
34             iOSWeatherWidget().create_weather()
35
36     #####
37
38     class CalendarWidget(ABC):
39         @abstractmethod
40         def create_calendar(self): ...
41
42     class iPadOSCalendarWidget(CalendarWidget):
43         def create_calendar(self):
44             print("Criando um calendário para iPadOS")
45
46     class iOSCalendarWidget(CalendarWidget):
47         def create_calendar(self):
48             print("Criando um calendário para iOS")
49
50     #####
51
52     class WeatherWidget(ABC):
53         @abstractmethod
54         def create_weather(self): ...
55
56     class iPadOSWeatherWidget(WeatherWidget):
57         def create_weather(self):
58             print("Criando um painel climático para iPadOS")
59
60     class iOSWeatherWidget(WeatherWidget):
61         def create_weather(self):
62             print("Criando um painel climático para iOS")
63
64     #####
65
66     class ApplicationInterface:

```

```

67     def get_factory(self, platform_type):
68         if platform_type == "iPadOS":
69             return iPadOSFactory()
70         if platform_type == "iOS":
71             return iOSFactory()
72
73         raise ValueError("Esta plataforma não existe")
74
75     #####
76
77     application_interface = ApplicationInterface()
78
79     # Platform type can be read from config file etc.
80     print("#"*40)
81
82     print("***** iPadOS *****")
83     widget_factory = application_interface.get_factory("iPadOS")
84     widget_factory.create_calendar()
85     widget_factory.create_weather()
86     print("#"*40)
87
88     print("***** iOS *****")
89     widget_factory = application_interface.get_factory("iOS")
90     widget_factory.create_calendar()
91     widget_factory.create_weather()
92     print("#"*40)
93
94     #####
95
96     from abc import ABC, abstractmethod
97
98     # =====
99     # The Abstract Factory is a creational design pattern that allows producing
100    # families
101    # of related or dependent objects without specifying their concrete classes.
102    #
103    # Abstract Factory classes are often based on a set of Factory Methods, but you
104    # can also use
105    # Prototype to compose the methods on these classes.
106    #
107    # For this lesson, we will use the Factory Methods approach
108    # =====
109    # 1) ABSTRAÇÕES (produtos) - o cliente conhece apenas isso
110    # =====

```

```

111
112 class DatasetLoader(ABC):
113     @abstractmethod
114     def load(self) -> None: ...
115
116 class Model(ABC):
117     @abstractmethod
118     def train(self) -> None: ...
119
120 class Visualizer(ABC):
121     @abstractmethod
122     def plot(self) -> None: ...
123
124 # =====
125 # 2) ABSTRAÇÃO DA FÁBRICA - contrato da família de produtos
126 # =====
127
128 class DataScienceFactory(ABC):
129     @abstractmethod
130     def create_dataset_loader(self) -> DatasetLoader: ...
131
132     @abstractmethod
133     def create_model(self) -> Model: ...
134
135     @abstractmethod
136     def create_visualizer(self) -> Visualizer: ...
137
138 # =====
139 # 3) PRODUTOS CONCRETOS - Família Local (Pandas / Sklearn / Matplotlib)
140 # =====
141
142 class PandasDatasetLoader(DatasetLoader):
143     def load(self) -> None:
144         print("[PandasDatasetLoader] Lendo CSV local com pandas.read_csv(...).
145             Amostras: 10_000")
146
147 class SklearnModel(Model):
148     def train(self) -> None:
149         print("[SklearnModel] Treinando LogisticRegression(solver=\"lbfgs\").
150             Acurácia de validação: 0.89")
151
152 class MatplotlibVisualizer(Visualizer):
153     def plot(self) -> None:
154         print("[MatplotlibVisualizer] Plotando curva ROC e matriz de confusão com
155             Matplotlib.")

```

```

154
155 # =====
156 # 4) PRODUTOS CONCRETOS - Família Distribuída (Spark / MLlib / Seaborn)
157 # =====
158
159 class SparkDatasetLoader(DatasetLoader):
160     def load(self) -> None:
161         print("[SparkDatasetLoader] Lendo dados no cluster:
162             spark.read.parquet(\"s3://bucket/dataset\"). Linhas: 120_000_000")
163
164 class MLlibModel(Model):
165     def train(self) -> None:
166         print("[MLlibModel] Treinando RandomForestClassifier em cluster (MLlib).
167             AUC de validação: 0.92")
168
169 class SeabornVisualizer(Visualizer):
170     def plot(self) -> None:
171         print("[SeabornVisualizer] Gerando pairplot e heatmap de correlação com
172             Seaborn.")
173
174 # =====
175 # 5) FÁBRICAS CONCRETAS - produzem uma FAMÍLIA coerente de produtos
176 # =====
177
178 class LocalPandasFactory(DataScienceFactory):
179     """Família "local" para dados pequenos/medianos."""
180     def create_dataset_loader(self) -> DatasetLoader:
181         print("[LocalPandasFactory] -> criando DatasetLoader (Pandas)")
182         return PandasDatasetLoader()
183
184     def create_model(self) -> Model:
185         print("[LocalPandasFactory] -> criando Model (Scikit-learn)")
186         return SklearnModel()
187
188     def create_visualizer(self) -> Visualizer:
189         print("[LocalPandasFactory] -> criando Visualizer (Matplotlib)")
190         return MatplotlibVisualizer()
191
192 class DistributedSparkFactory(DataScienceFactory):
193     """Família "distribuída" para grandes volumes de dados."""
194     def create_dataset_loader(self) -> DatasetLoader:
195         print("[DistributedSparkFactory] -> criando DatasetLoader (Spark)")
196         return SparkDatasetLoader()
197
198     def create_model(self) -> Model:
199         print("[DistributedSparkFactory] -> criando Model (MLlib)")

```

```

197         return MLlibModel()
198
199     def create_visualizer(self) -> Visualizer:
200         print("[DistributedSparkFactory] -> criando Visualizer (Seaborn)")
201         return SeabornVisualizer()
202
203     # =====
204     # 6) "SELETOR" DE FÁBRICA - ponto único de decisão concreta
205     #     (poderia vir de config/env/CLI; mantido simples para aula)
206     # =====
207
208     class ApplicationInterface:
209         def get_factory(self, stack: str) -> DataScienceFactory:
210             """
211             stack: "local" ou "distributed"
212             Retorna a fábrica apropriada, sem expor classes concretas ao restante do
213             cliente.
214             """
215             if stack == "local":
216                 print("[ApplicationInterface] Selecionando família LOCAL (Pandas/
217                     Sklearn/Matplotlib)")
218                 return LocalPandasFactory()
219             if stack == "distributed":
220                 print("[ApplicationInterface] Selecionando família DISTRIBUÍDA (Spark/
221                     MLlib/Seaborn)")
222                 return DistributedSparkFactory()
223             raise ValueError("Stack inválida. Use \"local\" ou \"distributed\".")
224
225     # =====
226     # 7) DRIVER CODE
227     # =====
228
229     app = ApplicationInterface()
230
231     # ---- Execução com a família LOCAL ----
232     print("\n==== Cenário A: Pipeline LOCAL (dados pequenos/medianos) =====")
233     factory = app.get_factory("local") # cliente recebe a FÁBRICA
234     (abstração)
235     loader = factory.create_dataset_loader() # abstração DatasetLoader
236     modelo = factory.create_model() # abstração Model
237     viz = factory.create_visualizer() # abstração Visualizer
238
239     print("\n[Cliente] Executando pipeline LOCAL (mesmo cliente, família 1):")
240     loader.load()
241     modelo.train()
242     viz.plot()

```

```

240
241 # ---- Troca de fábrica em tempo de execução (sem mudar o cliente) ----
242 print("\n==== Cenário B: Pipeline DISTRIBUÍDO (big data) =====")
243 factory = app.get_factory("distributed")           # troca a família
244 loader = factory.create_dataset_loader()
245 modelo = factory.create_model()
246 viz = factory.create_visualizer()
247
248 print("\n[Cliente] Executando pipeline DISTRIBUÍDO (mesmo cliente, família 2):")
249 loader.load()
250 modelo.train()
251 viz.plot()

```

aqui estamos criando um cara pra carregar dataset, rodar modelo, e visualizar resposta analítica
 diagnostica podia ser assim classe abstrata define um comportamento da class (modelo, dataset, visualizacao)

mais uma classe com 3 metodos, a fabrica cria as mesmas classes, com as funções definidas antes
 usuario so precisa conhecer as abstrações das nossas fábricas se usa muito mais o abstract
 factory do que o factory method (porque?) voce pode criar quantos loaders vcs quiser, quantos
 modelos vcs quiserem. a fabrica local de pandas tem a mesma estrutura do datascience factory pra
 mudar, você só precisa pegar a classe de Modelo. voce criou a classe inteira e só precisa mudar
 na linha de SkLearn a única decisão do usuário é escolher a fabrica que ele quer?

como estruturar essa quantidade de classes se vc mudar a estrutura vc muda o padrão o que
 pode mudar a vontade é acrescentar classes vc (não) pode mudar classes concretas

voltando ao exemplo da loja testes de regressão se eu quiser ter uma calculadora de frete e recibo
 diferente pra cada estado, o que fazer? se quiser evoluir, usa um abstractfactory nelas cria factory
 method na calculadora e no recibo, e o abstractfactory nos centrossingleton de fabrica defrete OCP
 se a gente quiser colocar belem no frete aereo

pra fazer isso, teriamos que estender de Frete, sem precisar modificar só estendendo. copia o
 fretefluvial e bota aereo